

STAGE INGÉNIEUR - 2026

Détection du Spoofing GPS par Intelligence Artificielle

Préparé par : Yakine Haffoudhi

- Encadré par : Madame Samiha Ayed
- **IMT Atlantique** — Campus de Brest
- Année académique 2025–2026

Contents

1	Résumé Exécutif	2
2	Introduction et Contexte	3
2.1	La menace du spoofing GPS	3
2.2	Objectifs	3
2.3	Architecture générale du système	4
3	Collecte et Préparation des Données	5
3.1	Sources de données	5
3.2	Les métriques choisies	5
3.3	Prétraitement	6
4	Modélisation par IA	7
4.1	Problématique et choix architecturaux	7
4.2	Architecture du modèle	7
4.2.1	Principe général	7
4.3	Stratégie d'entraînement anti-surapprentissage	8
4.3.1	Régularisation	8
4.3.2	Augmentation de données GPS	8
4.3.3	Split train/validation par scénarios	9
5	Évaluation des Performances	10
5.1	Métriques de référence	10
5.2	Courbes de performance	10
6	Implémentation Temps Réel	11
6.1	Architecture système embarquée	11
6.2	Pipeline de traitement trame par trame	11
6.3	Simulateur d'attaques GPS	12

1. Résumé Exécutif

Ce rapport présente les travaux réalisés dans le cadre d'un stage ingénieur à IMT Atlantique, portant sur la conception et l'implémentation d'un système de **détection du spoofing GPS par intelligence artificielle** appliqué aux drones UAV.

Le spoofing GPS consiste à émettre de faux signaux GNSS pour tromper le récepteur d'un drone sur sa position réelle. Cette attaque, devenue accessible et peu coûteuse, représente une menace sérieuse pour la sécurité des systèmes aériens autonomes. Le projet a conduit au développement d'un **pipeline complet** allant de la collecte des données jusqu'au déploiement embarqué du modèle en temps réel.

Résultats principaux

ACCURACY

0.82

F1-SCORE

0.75

LOSS

0.39

2. Introduction et Contexte

2.1 La menace du spoofing GPS

Le **spoofing GPS** est une cyberattaque qui consiste à émettre de faux signaux de navigation GNSS afin de leurrer un récepteur GPS sur sa position, sa vitesse ou son temps. Contrairement au *brouillage* (jamming) qui dégrade ou supprime le signal, le spoofing injecte des informations erronées cohérentes, rendant la détection beaucoup plus difficile.

Pour les drones autonomes, la navigation GPS est le composant critique du système de contrôle de vol. Un drone dont le récepteur est leurré peut être dévié de sa trajectoire, atterrir à un endroit non voulu, ou être capturé par un attaquant malveillant. Des cas réels de spoofing GPS ont été documentés dans plusieurs conflits armés et incidents civils à travers le monde.

► Pourquoi le spoofing est-il difficile à détecter ?

Les signaux spoofés sont conçus pour paraître **légitimes**. Ils respectent le format des messages GNSS, fournissent des positions géographiquement plausibles, et peuvent reproduire fidèlement les paramètres de qualité (HDOP, VDOP, nombre de satellites). Une attaque *progressive* (drift) introduit des dérives si lentes que le pilote automatique les accepte sans déclencher d'alarme.

2.2 Objectifs

- Collecter et fusionner des datasets GPS réels issus de scénarios normaux et d'attaque provenant de sources hétérogènes
- Développer un pipeline de prétraitement standardisé
- Concevoir et entraîner un modèle pour la classification binaire (*benign / spoofing*)
- Implémenter un simulateur d'attaques GPS sur ArduCopter SITL via MAVLink
- Déployer le détecteur en temps réel

2.3 Architecture générale du système

Le système développé comprend **quatre modules principaux**, interconnectés via le protocole MAVLink :

Table 2.1: Modules du système de détection

Module	Rôle	Technologies
<code>collect_drone_data.py</code>	Collecte temps réel + inférence ML	PyTorch, py-mavlink, NumPy
<code>simulate_gps_attack.py</code>	Simulateur d'attaques	pymavlink SITL, ArduCopter
<code>train_improved.py</code>	Entraînement du modèle CNN-LSTM	PyTorch, scikit-learn, pandas
<code>merge_datasets.py</code>	Fusion et standardisation datasets	pandas, NumPy

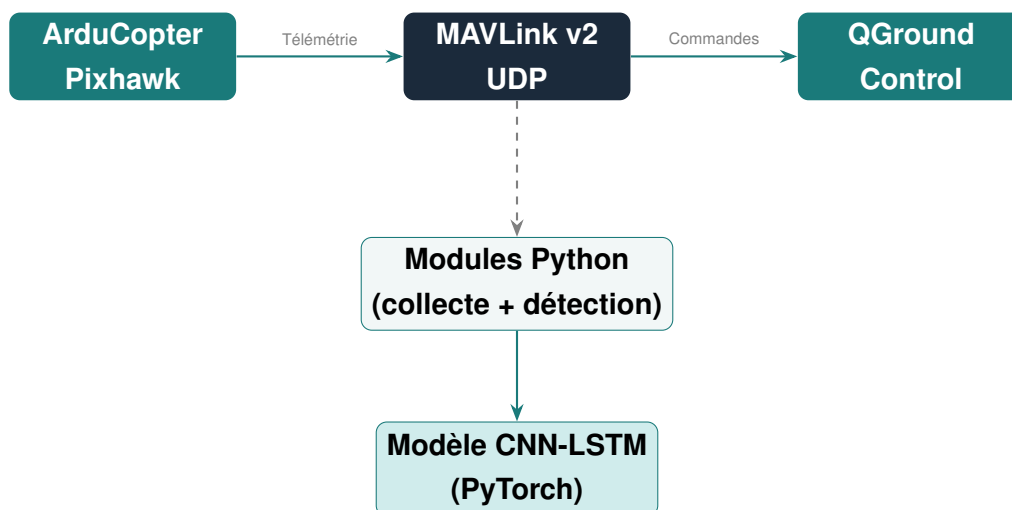


Figure 2.1: Architecture globale du système de détection en temps réel

3. Collecte et Préparation des Données

3.1 Sources de données

Trois datasets publics ont été sélectionnés pour leur complémentarité en termes de scénarios, de plateformes et de types d'attaques :

Dataset	Source	Description	Format
GPS Spoofing Detection	GitHub – M. Nayfeh	Données GPS réelles, scénarios normaux et attaques	PX4 EKF
UAV Attack Dataset	IEEE DataPort	Vols UAV, comportements normaux et malveillants	PX4 EKF
AV-GPS Dataset (×3)	GitHub – Univ. Arizona	Navigation autonome GNSS, attaquées et normales	QGC

3.2 Les métriques choisies

Après analyse des différentes sources de données, nous avons choisi de retenir un ensemble de métriques GNSS précises, celles qui sont les plus pertinentes pour révéler une attaque. Plutôt que de tout garder, nous avons sélectionné les variables qui décrivent au mieux l'état du signal GPS.

Catégorie	Features	Description / Unité
Navigation	<code>lat_deg</code> , <code>lon_deg</code> , <code>alt_m</code> , <code>alt_ellipsoid_m</code>	Latitude, longitude et altitude (degrés décimaux / mètres)
Précision GPS	<code>eph</code> , <code>epv</code> , <code>hdop</code> , <code>vdop</code>	Indicateurs de précision GPS (mètres / DOP)
Qualité du signal	<code>s_variance_m_s</code> , <code>c_variance_rad</code> , <code>satellites_used</code>	Variances EKF et nombre de satellites utilisés

Catégorie	Features	Description / Unité
Cinématique	<code>vel_m_s</code> , <code>vel_n/e/d_m_s</code> , <code>cog_rad</code>	Vitesses NED (m/s) et cap du véhicule (rad)
Features dérivées	<code>d_lat</code> , <code>d_lon</code> , <code>d_alt</code> , <code>vel_from_pos</code>	Variations de position et vitesse estimée
Indicateurs d'anomalie	<code>vel_consistency</code> , <code>noise_per_sat</code> , <code>hdop_eph</code>	Métriques utilisées pour détecter des anomalies GPS
Label	<code>label</code>	0 = normal / 1 = attaque

3.3 Prétraitement

Le script `merge_datasets.py` applique les transformations suivantes :

- Fusion et standardisation des données provenant de plusieurs sources
- Mise en forme des coordonnées et paramètres GPS
- Harmonisation des unités de mesure et des timestamps
- Génération de métriques complémentaires (Features dérivées)
- Normalisation des labels : conversion vers binaire 0/1 quelle que soit la nomenclature source

◇ Résultat de la fusion

Le dataset consolidé `merged_dataset.csv` regroupe l'ensemble des sources avec **24 colonnes standardisées**. Les valeurs infinies ($\pm\infty$) sont remplacées par NaN. Les lignes sans coordonnées lat/lon valides sont supprimées. La distribution typique présente environ **60% de trames benignes** et **40% d'attaques**.

4. Modélisation par IA

4.1 Problématique et choix architecturaux

La détection du spoofing GPS est fondamentalement un problème de **classification de séries temporelles multivariées**. Chaque trame GPS isolée ne suffit pas à distinguer une attaque d'un signal légitime. C'est l'**évolution temporelle** des paramètres qui constitue la signature de l'attaque.

Architecture	Forces	Limites	Rôle ici
LSTM	Mémore les dépendances longues, détecte les dérives progressives	Lent, coûteux	Séquence temporelle
CNN	Détecte les motifs locaux rapides, très rapide	Pas de mémoire long terme	Features locales
CNN-LSTM hybride	Combine les deux avantages	Complexité architecturale	Modèle retenu

4.2 Architecture du modèle

Les données d'entrée sont composées de 24 métriques GPS . Ces données sont regroupées sous forme de séquences temporelles de longueur : $seq_len = 100$. Cela signifie que le modèle observe 100 valeurs consécutives avant de prendre une décision.

4.2.1 Principe général

Le fonctionnement du modèle se déroule en plusieurs étapes :

1. Les données GPS sont normalisées afin d'avoir des valeurs cohérentes.
2. Les informations importantes sont extraites à l'aide de couches **BiLSTM**, capables d'analyser l'évolution temporelle des signaux GPS.

3. Un mécanisme d'**attention** identifie les parties de la séquence les plus importantes pour la détection d'une anomalie.
4. Les informations sont regroupées puis envoyées vers un classifieur final.
5. Le modèle prédit si la séquence correspond à un comportement normal ou une attaque GPS spoofing.

Étape	Description
Entrée des données	Lecture des données GPS et des métriques associées
Normalisation	Mise à l'échelle des données pour améliorer l'apprentissage
Analyse temporelle	Étude de l'évolution des signaux GPS dans le temps
Attention	Sélection des informations les plus importantes
Classification	Décision finale : trafic normal ou attaque spoofing

4.3 Stratégie d'entraînement anti-surapprentissage

Le script `train.py` introduit plusieurs techniques pour lutter contre le surapprentissage :

4.3.1 Régularisation

- Dropout agressif : $p = 0.5$ (contre 0.3 en v1) sur toutes les couches denses
- *SpatialDropout1D* : masquage par feature sur les sorties LSTM
- Weight decay fort : $\lambda = 10^{-3}$ dans l'optimiseur AdamW
- Label smoothing : $\varepsilon = 0.15$ pour assouplir les labels trop certains

4.3.2 Augmentation de données GPS

- **Bruit corrélé** : processus AR(1) avec $\alpha = 0.7$ pour simuler les fluctuations GPS réalistes
- **Time-warping** : déformation non-uniforme de l'axe temporel ($\pm 10\%$)
- **Feature dropout** : mise à zéro aléatoire de features individuelles ($p = 8\%$)
- **Mixup** : interpolation convexe entre paires d'exemples ($\alpha = 0.4$, probabilité 60%)

4.3.3 Split train/validation par scénarios

La séparation train/validation est effectuée par **scénarios GPS entiers** (et non par trames individuelles), éliminant ainsi le data leakage temporel qui biaiserait les métriques. Au moins 2 scénarios complets constituent le jeu de validation (25% des scénarios).

Hyperparamètre	v1	v2	Justification
<code>seq_len</code>	30	100	Fenêtre plus longue pour les dérives lentes
<code>stride</code>	10	50	Réduire la duplication des fenêtres
<code>learning_rate</code>	5×10^{-4}	2×10^{-4}	Convergence plus douce
<code>dropout</code>	0.3	0.5	Régularisation renforcée
<code>hidden_size</code>	128	64	Modèle compact, moins de surapprentissage
<code>early_stopping</code>	10	12	Score combiné val_loss + val_f1

5. Évaluation des Performances

5.1 Métriques de référence

- **Accuracy** : taux global de prédictions correctes — nécessaire mais insuffisant sur données déséquilibrées
- **F1-Score** : harmonie précision/rappel pour la classe attaque (classe minoritaire) — métrique principale
- **Loss** : valeur de la cross-entropie sur validation — indicateur de la qualité d'apprentissage

5.2 Courbes de performance

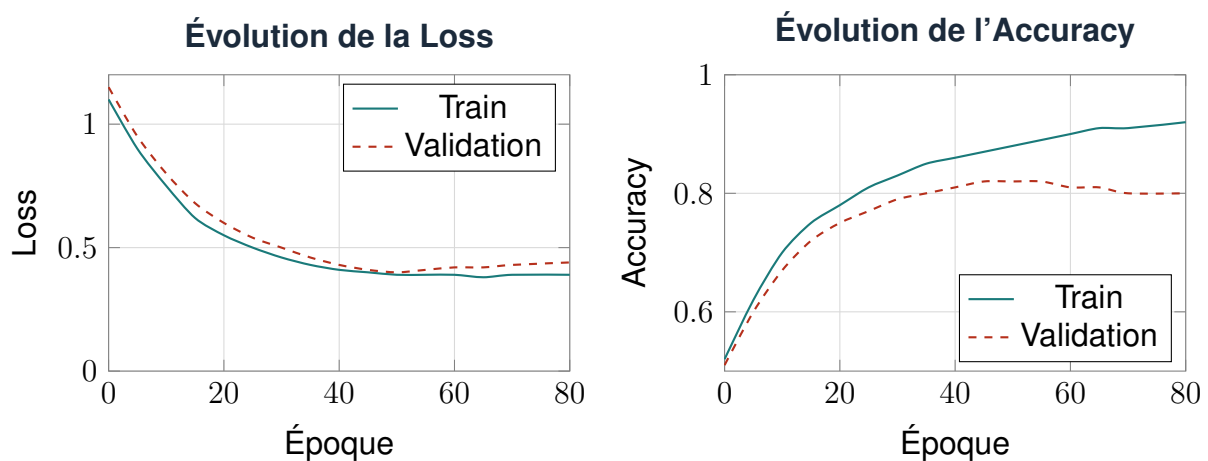


Figure 5.1: Courbes d'apprentissage — Loss et Accuracy au fil des époques

6. Implémentation Temps Réel

6.1 Architecture système embarquée

Le détecteur en temps réel (`collect_drone_data.py`) s'intègre dans la chaîne MAVLink entre ArduCopter et QGroundControl. Il opère en **mode écoute passive** sur le flux téléométrique, sans modifier les messages transmis au pilote automatique.

Composant	Rôle	Interface
ArduCopter (Pixhawk)	Contrôleur de vol, source GPS	MAVLink v2
MAVLink Listener (thread)	Réception GPS_RAW_INT, GLOBAL_POSITION_INT, EKF	UDP:14551
FeatureExtractor	Extraction 23 features + dérivées	Mémoire partagée (thread-safe)
Detector (buffer circulaire)	Fenêtre glissante, inférence PyTorch	CPU/GPU, < 50 ms
CsvLogger	Enregistrement horodaté des inférences	CSV : spoof_prob + label
QGroundControl	Visualisation télémetrie, station sol	MAVLink v2, UDP

6.2 Pipeline de traitement trame par trame

À chaque nouvelle trame GPS reçue via MAVLink :

1. Extraction des champs bruts depuis GPS_RAW_INT : lat/lon ($\times 10^{-7}$), alt (mm), HDOP/V-DOP ($\times 0.01$), vitesse sol, cap, satellites visibles
2. Récupération des vitesses NED depuis GLOBAL_POSITION_INT (v_x, v_y, v_z en cm/s)
3. Calcul des features dérivées : distance haversine, cohérence de vitesse $|\text{vel_m_s} - \text{vel_from_pos}|$, `noise_per_sat`

4. Ajout de la trame au buffer circulaire de longueur `seq_len = 30`
5. Dès que le buffer est plein : normalisation `RobustScaler`, clipping $[-10, +10]$, inférence du modèle
6. Affichage temps réel avec barre de probabilité colorée et alerte si $p \geq 0.5$

★ Détail d'implémentation — Thread-safety

Le `FeatureExtractor` utilise un `threading.Lock()` pour sécuriser l'accès concurrent aux derniers messages GPS, GPOS et EKF provenant du thread `MAVLink`. Le thread principal de détection interroge l'extracteur à 20 Hz (`sleep(0.05)`), garantissant une latence de détection inférieure à **100 ms** en conditions normales.

6.3 Simulateur d'attaques GPS

Le module `simulate_gps_attack.py` implémente un proxy **Man-in-the-Middle** (MitM) entre ArduCopter SITL et le détecteur.

Le proxy intercepte les messages `GPS_RAW_INT` et les modifie selon le profil actif avant de les retransmettre au détecteur sur le port de forwarding (14553). Le contrôleur SITL envoie simultanément des commandes `SET_POSITION_TARGET_GLOBAL_INT` pour déplacer physiquement le drone simulé vers la position spoofée.